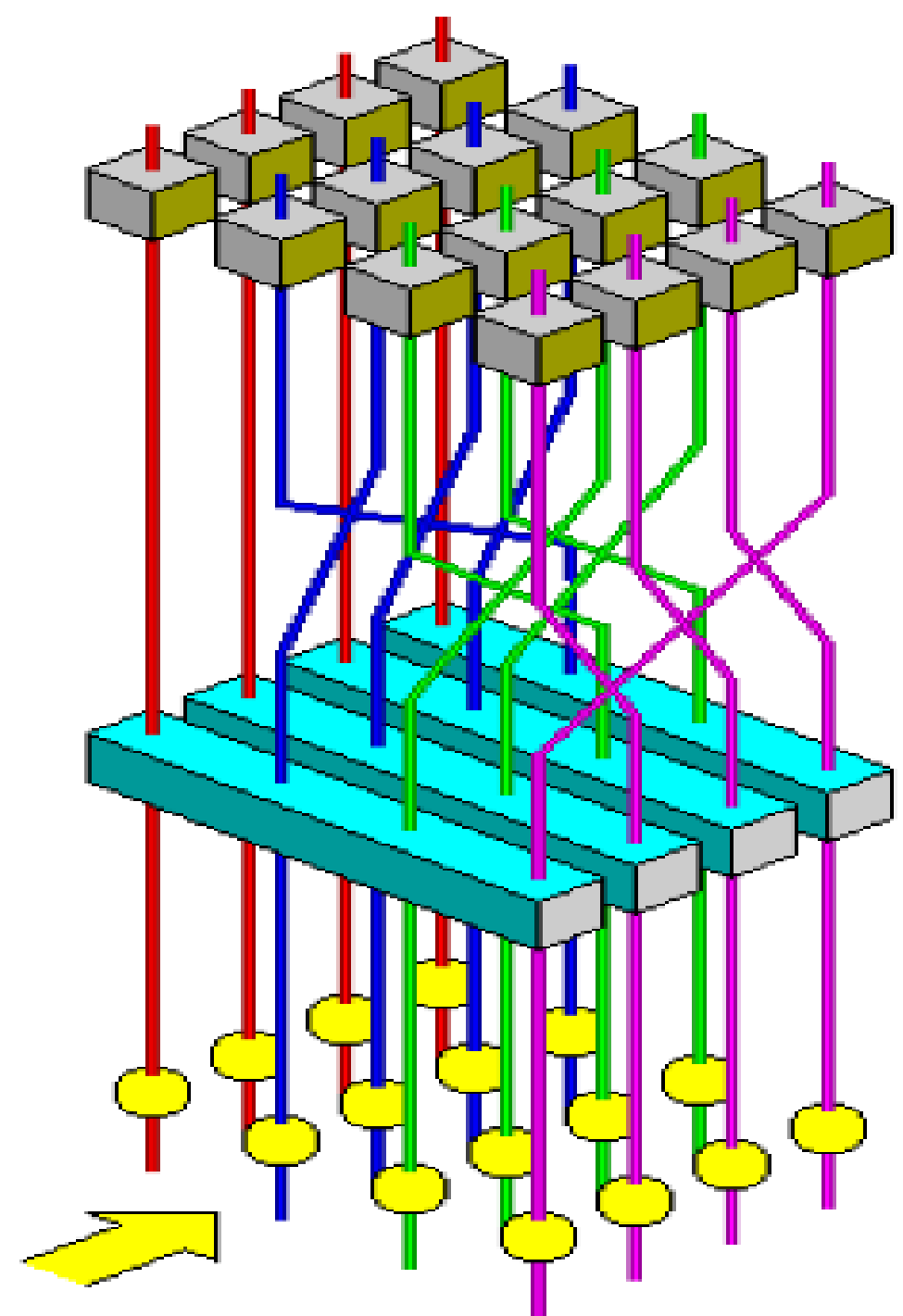


Processor Design Project

Group 24

12-06-202
6

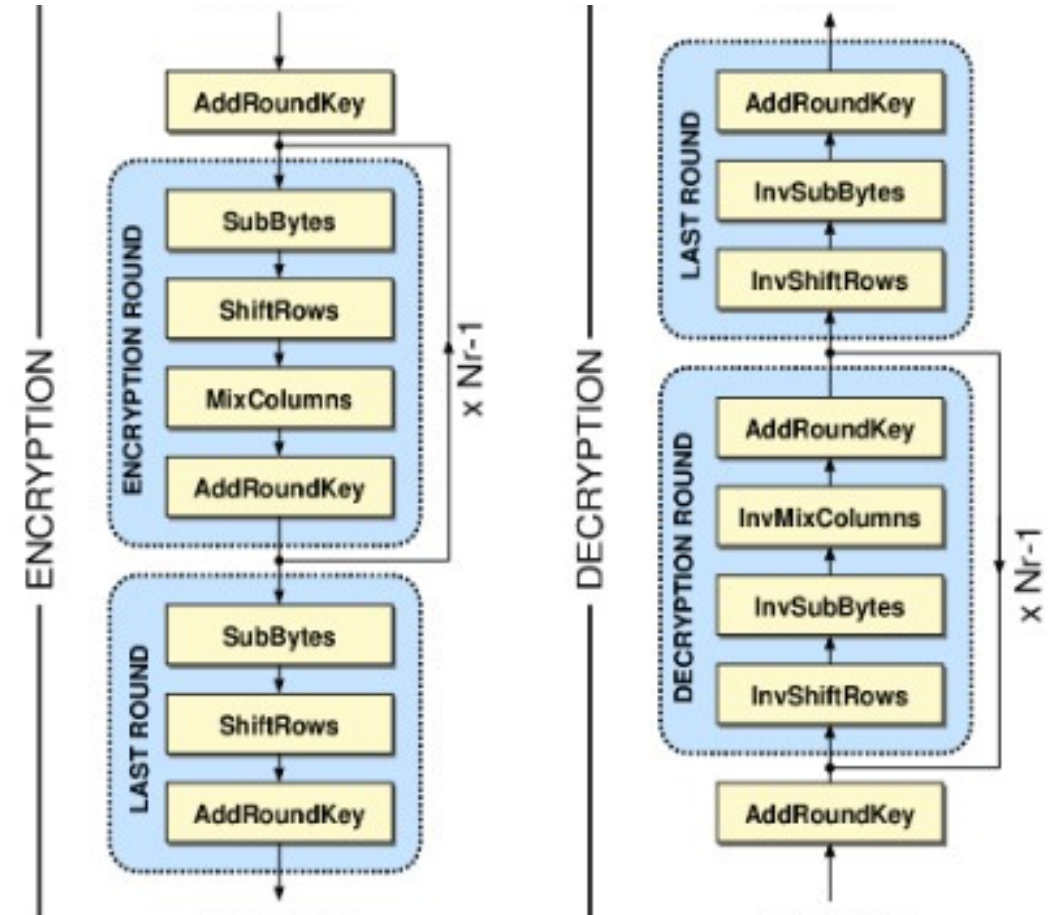
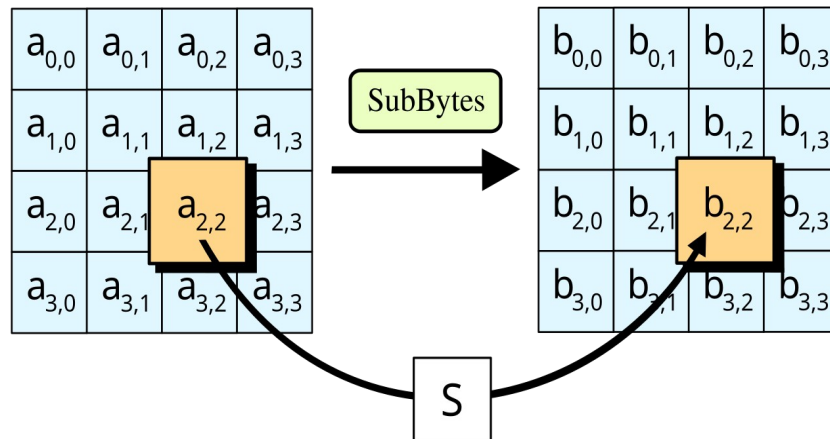


Context: Why AES is needed?

Advanced Encryption Standard (AES)

What is AES?

- Symmetric block cipher — same key for encryption and decryption
- Operates on a **4×4 matrix of bytes** called the **state** (= 128 bits)
- Key is expanded into **11 round keys** via the key schedule



Advanced Encryption Standard (**AES**)

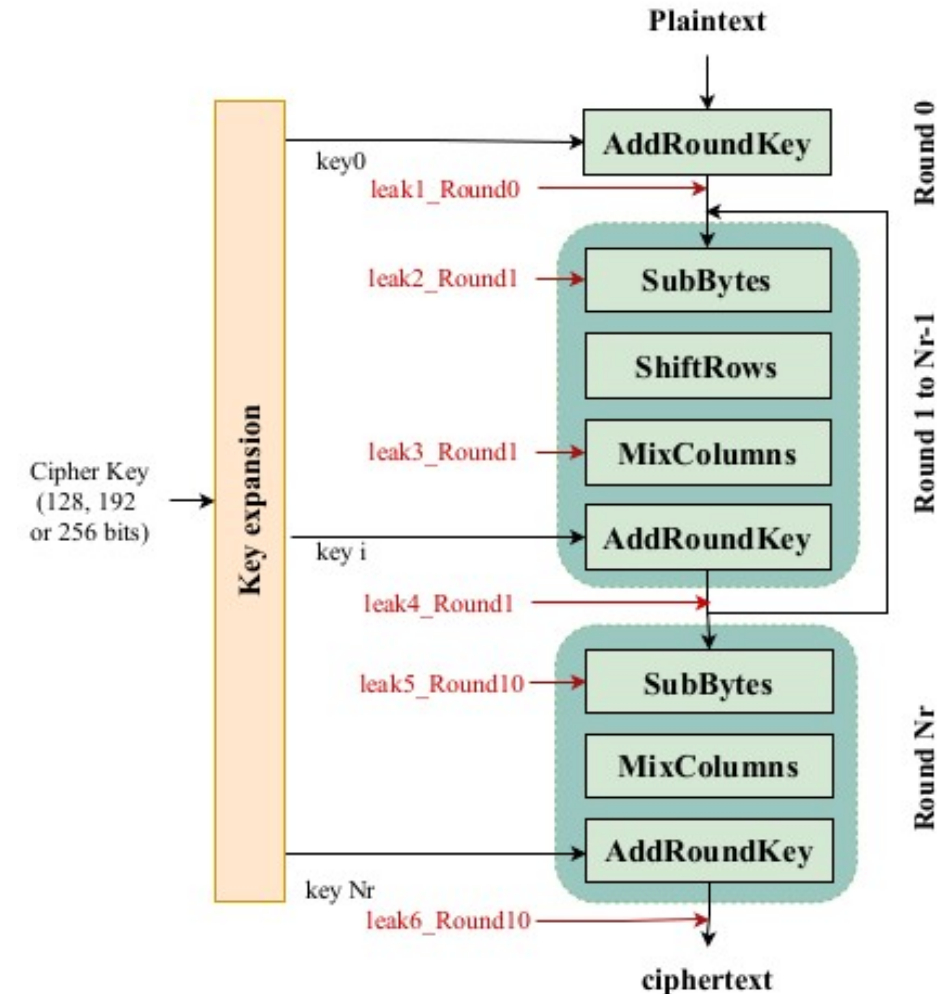
How does AES work?

Initial Round Plain text XOR'd with round key 0

Middle Rounds × 9

- **SubBytes** — Byte substitution via S-box (confusion)
- **ShiftRows** — Cyclic row shifts to spread data across columns
- **MixColumns** — Column mixing over $GF(2^8)$ (diffusion)
- **AddRoundKey** — XOR with current round key

Final Round (Round 10) Same as middle rounds but **no MixColumns**



Advanced Encryption Standard (**AES**)



Where is AES used?

- **HTTPS/TLS** — Every secure website visit (banking, shopping) encrypts data using AES
- **Wi-Fi (WPA2/WPA3)** — Your home/office network uses AES to protect wireless traffic
- **File encryption** — BitLocker (Windows), FileVault (Mac) encrypt your hard drive with AES
- **Cloud storage** — Google Drive, Dropbox encrypt stored files using AES-256
- **VPNs** — All major VPN providers (NordVPN, OpenVPN) use AES as their core cipher
- **USB drives & SSDs** — Hardware-encrypted storage uses dedicated AES engines
- **Smart cards & passports** — AES protects biometric data in modern ID documents
- **IoT devices** — Sensors and embedded systems use AES for secure communication

Baseline software implementation

- **Pure C implementation** — using standard loads, stores, XORs, shifts

Resource	Count
Cycle Count	61184
LUTs	4644
Registers	2163
Muxes	284

Baseline Bottlenecks

Function	Cycles	% of Total
<code>mix_columns</code>	50,643	83.8%
<code>add_round_key</code>	2,850	4.7%
<code>sub_bytes</code>	2,679	4.4%
<code>expand_key</code>	2,417	4.0%
<code>shift_rows</code>	510	0.8%

How does hardware help?

- Eliminates all the `gf_mult` inlining (the 186-instruction `mix_columns` becomes a handful of `aes32esmi` calls)
- Operates entirely on registers, with no BRAM loads for the S-box or MixColumn
- `aes32esmi` combines SubBytes + ShiftRows + MixColumns + AddRoundKey into **a single hardware instruction** that completes in **1 cycle**

Performance after implementation of AES hardware support

aes32esmi does **SubBytes + ShiftRows + MixColumns + AddRoundKey** in a single cycle, removing the **mix_columns** bottleneck that was 83.8% of software runtime.

Version	Cycles	vs software baseline
Software AES (baseline)	59,560	1.0×
+ aes32esmi / aes32esi hardware	6,260	9.5× faster

The looped hardware version (6,260) is the starting point for the later optimisations: custom loop-unroll pass → 4,800, parallel DOM S-box → 4,104 (covered on the compiler and RTL slides).

Vulnerabilities to side channel attacks

The Zkne S-box and MixColumns switch a data-dependent number of bits each cycle, so the instantaneous power draw leaks the byte being processed.

Correlation Power Analysis exploits this: correlating measured power against $\text{HW}(\text{SBOX}[\text{pt XOR guess}])$ makes the correct key guess separate from the other 255. On our unprotected core a key byte falls in about 100 traces.

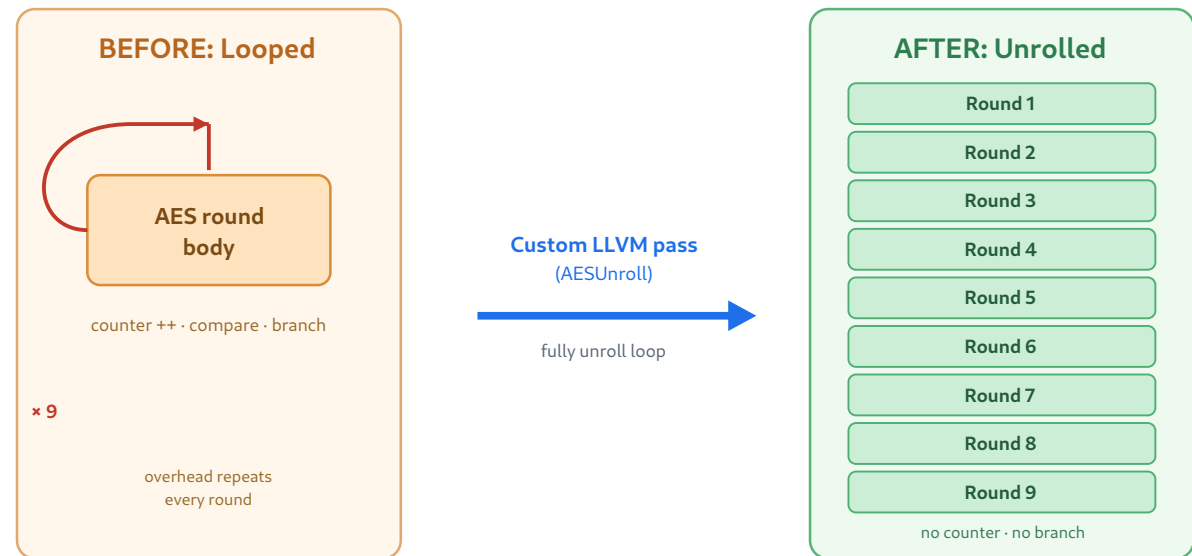
Countermeasure: Domain-Oriented Masking (DOM). Every secret is split into two random shares that recombine only inside registered DOM-AND gates, so no single wire (and no glitch) correlates with the real value.

Design follows the TU Delft secure-Zkne paper of Kassimi, Aljuffri, Hamdioui and Taouil (2026).

Compiler Optimizations: Custom LoopUnroll Pass

What was done?

- Wrote our own LLVM compiler pass to fully unroll the 9-round AES loop
- Calls LLVM's unroll routine directly instead of using a built-in flag
- Flattens all 9 rounds into straight line code
- Detects the loop by spotting the AES instructions inside it, so it survives inlining



Compiler Optimizations: Custom LoopUnroll Pass

Why we did it?

- The looped version wastes cycles on loop overhead every round (counter, branch, condition check, key pointer reload).
- Built in unrolling is cost model driven and often refuses to unroll, especially when optimizing for size.
- Wanted full control to force unrolling on exactly the target loop.
- Goal was to build the optimization myself, not just flip a compiler switch.

Compiler Optimizations: Custom LoopUnroll Pass

Results

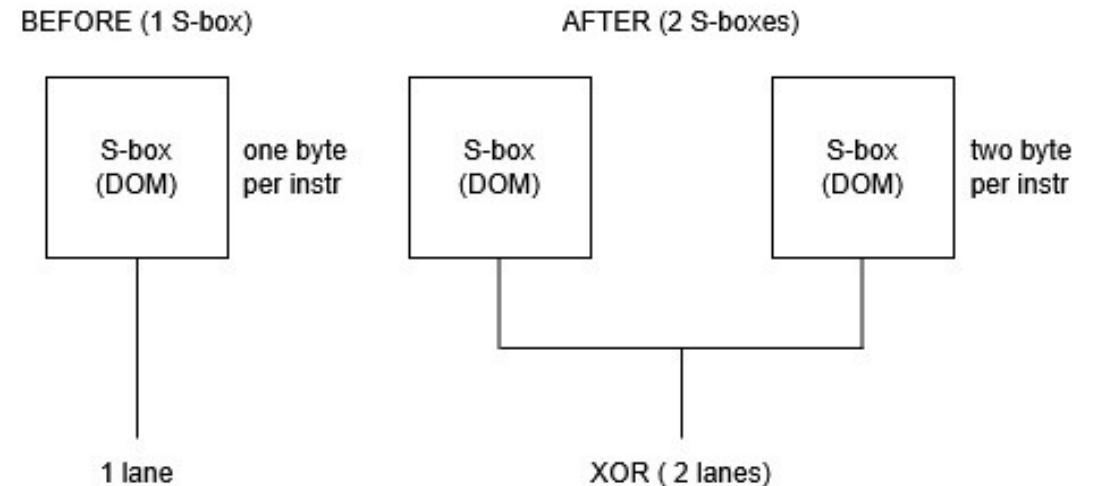
- Brought the software from 6,260 to 4,800 cycles, about 23 percent faster
- Same correct ciphertext, guarantees the loop is always fully unrolled

Build	Unroll Method	Cycles	Vs. Baseline
GCC -Os (looped)	none	6260	-
GCC -Os + #pragma unroll	Source pragma	6067	-3%
GCC -O2 + - funroll-loops	Build-in flag	5509	-12%
Clang -Os + Custom pass	AESUnroll	4800	-23%

RTL Optimization: Parallel DOM S-box

What was done?

- Added a custom instruction that runs two DOM protected AES S-boxes in parallel
- Each instruction now processes two-byte lanes of a column instead of one
- Built a new execution unit in the core (two S-boxes side by side) plus decoder and pipeline changes
- Software XORs the two halves together to finish each column
- Kept the original single S-box path intact as a fallback



RTL Optimization: Parallel DOM S-box

Why we did it?

- The DOM S-box is the real bottleneck: every byte pays a fixed 5 cycle latency and stalls the whole pipeline
- The original design did the S-box one byte at a time, so the 16 bytes per round were fully serial
- Running two S-boxes at once halves the number of instructions per round
- The masking (side channel protection) had to stay fully intact

RTL Optimization: Parallel DOM S-box

Results

Stage	Cycles	Vs. Original
Original	6260	baseline
Custom Unroll Pass	4800	23% faster
Parallel DOM S-box	4104	34% faster

Resource	Count
Cycle Count	4121
LUTs	11987
Registers	10412
Muxes	289
DSP	5

Validation Framework: Simulation-Based Power Analysis

How we proved the masking works, in simulation and on hardware

Power proxy: a SystemVerilog testbench drives plaintexts through the hardware S-box in Vivado xsim and logs the Hamming weight of the captured result register per trace.

One harness, two S-boxes: the same rig runs against the unprotected and the DOM-masked S-box, producing identical CSVs. The only variable is the design, so the before/after is clean.

Two standard attacks, scored in Python (numpy/scipy):

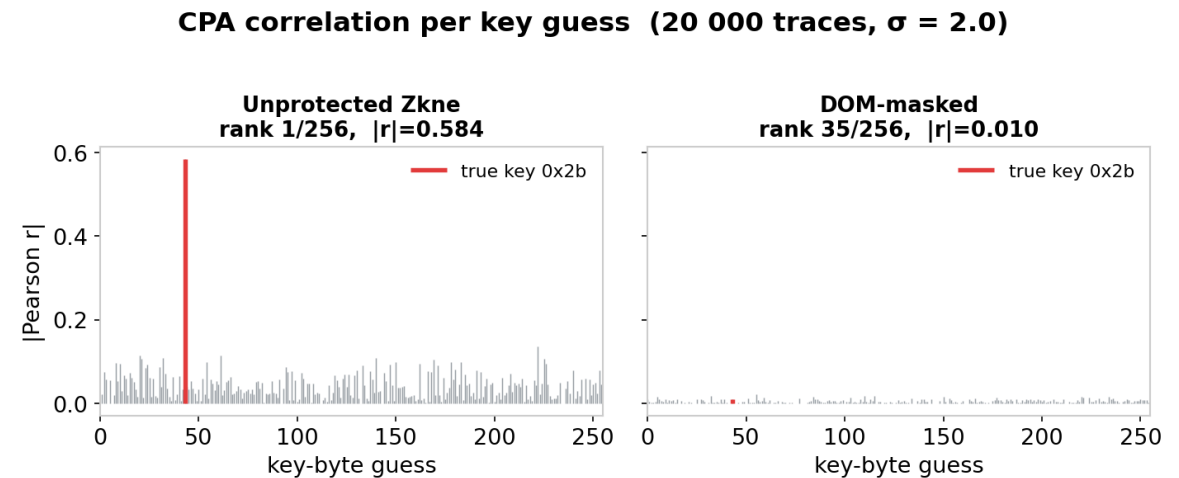
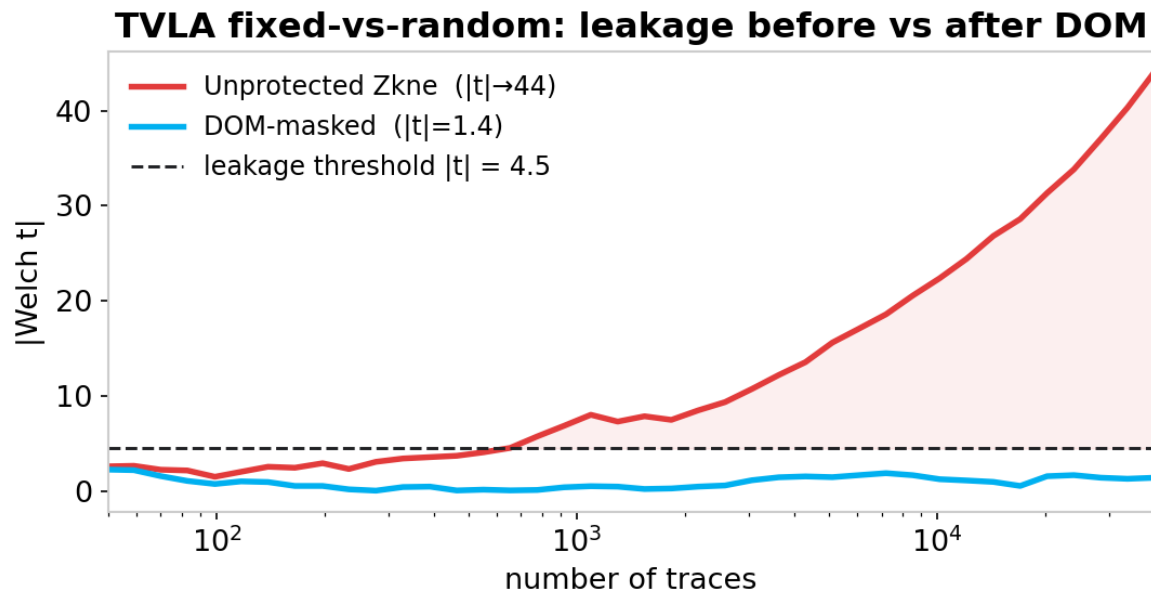
CPA: recover the key byte by correlating power with $\text{HW}(\text{SBOX}[\text{pt XOR guess}])$ over all 256 guesses, 20 000 traces.

TVLA: Welch fixed-vs-random t-test on 40 000 traces; $|t| > 4.5$ means first-order leakage (ISO/IEC 17825).

Why simulation first: it isolates the register's switching with no analog noise, giving fast feedback in the RTL loop, alongside Rishi's CW305 hardware validation.

Validation Results: DOM Defeats the Attack

Same rig, same noise ($\sigma = 2.0$). The only change is the S-box, and the leakage disappears.



Metric	Unprotected	DOM-masked
CPA true-key rank	1 / 256	35 / 256
CPA top $ r $	0.58	0.022
TVLA max $ t $	44.0	1.4
Crosses $ t = 4.5$?	Yes (9.8x)	No (0.3x)

Future Work: The Super Instruction

The observation:

- Same operation four times
- This results in:
 - Additional execution overhead
 - Repeated instruction fetch and decode.
 - Increased instruction count. .
- Independent, highly regular pattern.

The Super Instruction

The Idea:

- i) Introduce a super instruction
- ii) Replicate AES32/Zkne datapath four times
- iii) Execute in parallel within a single instruction.
- iv) Combine the results internally
and write back the final 32-bit result.

The Super Instruction

Architectural Changes

i) Introduced new opcodes and instruction formats

- New instruction encoding includes four source registers:
rs4 | rs3 | rs2 | rs1 | rd | opcode
- Single instruction to specify all operands

ii) Modified the register file

- Added an additional read port.
- Allows all operands to be fetched within a single instruction cycle.

The Super Instruction

iii) Replicated the Zkne hardware four times

- Instantiated four AES32/Zkne datapaths.

iv) Added Software support for these super instructions:

The Super Instruction

Limitations:

- i) Haphazard allocation of new opcodes. Might mess with the RISC-V convention in the long run
- ii) Did not want to modify the register file architecture for just 2 instructions.
- iii) Modest benefit, but high overhead (in terms of utilisation).

THANK YOU !!

